

W10D02 — Multi-service Compose + Networking

ML Engineer Journey · Phase 3 — Docker & Clean Architecture · 12 Mei 2026

1. Analoqi — Gedung Perkantoran

Bayangkan sebuah gedung perkantoran. Di dalamnya ada banyak ruangan — Ruang API, Ruang Database, Ruang Cache. Setiap penghuni cukup menyebut nama ruangan untuk berkomunikasi. Tidak perlu tahu nomor pintu (IP address) karena ada resepsionis internal yang mengurus routing.

Sekarang bayangkan dua gedung terpisah. Penghuni Gedung A tidak bisa menelepon Ruang Redis di Gedung B hanya dengan nama — butuh alamat eksternal lengkap, atau jembatan khusus antar gedung.

Docker networking bekerja persis seperti itu. Setiap docker-compose up membangun satu gedung (network). Semua service di dalamnya bisa saling panggil via nama. Dua Compose file = dua gedung terpisah = perlu custom network untuk menjembatani.

2. Cara Docker Networking Bekerja

Docker menggunakan **Linux bridge network** — sebuah virtual switch yang menghubungkan container-container dalam satu network. Setiap container mendapat IP internal, tapi yang lebih penting: Docker menjalankan **embedded DNS server** di 127.0.0.11 yang me-resolve service name ke IP container.

Tiga langkah yang terjadi saat container **api** memanggil **redis**:

- ① Container **api** melakukan DNS lookup untuk hostname **redis**
- ② Docker embedded DNS menjawab dengan IP internal container redis
- ③ Packet dikirim melalui bridge network — redis menerima request

Tidak ada hardcoded IP. Tidak ada konfigurasi manual. Hanya nama service yang digunakan sebagai hostname. Ini yang membuat Compose networking begitu bersih dan portable.

3. Default Network vs Custom Named Network

	Default Network	Custom Named Network
Dibuat otomatis?	Ya, saat docker-compose up	Tidak — harus deklarasi manual
Service name sebagai DNS?	Tidak	Ya
Isolasi antar Compose file?	Tidak — terpisah penuh	Bisa di-share via external: true
Container Compose lain bisa join?	Tidak	Ya
Nama network	<project>_default	Sesuai yang didefinisikan
Cocok untuk?	Dev sederhana, 1 Compose file	Multi-Compose, staging complex

4. docker-compose.yml — Multi-service (api + redis)

```
version: "3.9"

services:

  api:
    build: .
    container_name: risk_api
```

```

ports:
  - "8000:8000"
environment:
  - REDIS_HOST=redis      # service name, bukan IP – DNS resolve otomatis
  - REDIS_PORT=6379
volumes:
  - ./logs:/app/logs      # bind mount untuk dev logging
depends_on:
  - redis                  # api start SETELAH redis container ready
restart: unless-stopped

redis:
  image: redis:7-alpine    # alpine: ~30MB vs ~110MB non-alpine
  container_name: risk_redis
  ports:
    - "6379:6379"          # opsional: expose ke host untuk debug
  volumes:
    - redis-data:/data      # named volume: data persist meski restart
  restart: unless-stopped
  command: redis-server --appendonly yes    # AOF persistence

volumes:
  redis-data:                # Docker-managed, tidak terhapus saat down

```

Penjelasan baris kritis:

REDIS_HOST=redis — Kamu tidak menulis IP. Kamu menulis nama service. Docker DNS akan translate 'redis' ke IP internal container Redis saat runtime. Kalau nama service diganti, environment variable ini harus ikut ganti.

depends_on: - redis — Memastikan Docker start Redis dulu sebelum API. Tapi hanya memastikan container *started*, bukan *healthy*. Redis bisa belum siap menerima koneksi ketika API langsung mencoba connect. Solusi production: healthcheck (W10D05).

redis:7-alpine — Alpine Linux base image. Size ~30MB vs ~110MB untuk non-alpine. Aplikasi langsung dari prinsip image optimization (W09D05).

--appendonly yes — Redis AOF persistence mode. Setiap write dicatat ke file di dalam volume. Tanpa ini, restart container = semua data hilang.

redis-data:/data — Named volume. Redis menyimpan data ke /data secara default. Dengan volume ini, data survive saat container di-restart atau di-recreate.

5. Cara API Berkomunikasi dengan Redis

```

# app/services/cache_service.py
import os
import redis

REDIS_HOST = os.getenv("REDIS_HOST", "localhost") # fallback untuk dev tanpa Docker
REDIS_PORT = int(os.getenv("REDIS_PORT", 6379))

redis_client = redis.Redis(
    host=REDIS_HOST,          # di dalam Docker: "redis" (service name)
    port=REDIS_PORT,
    decode_responses=True     # return str, bukan bytes
)

def set_cache(key: str, value: str, ttl: int = 300) -> None:

```

```
redis_client.setex(key, ttl, value)
```

```
def get_cache(key: str) -> str | None:  
    return redis_client.get(key)
```

Kenapa `os.getenv('REDIS_HOST', 'localhost')`? Fallback 'localhost' membuat kode ini bisa dijalankan di dua mode: tanpa Docker (dev lokal) dan dalam Docker Compose. Satu kode, dua environment. Tidak ada if IS_DOCKER yang jelek.

6. Langkah Verifikasi Komunikasi Antar Service

Langkah	Perintah	Expected Output
① Jalankan compose	<code>docker-compose up -d</code>	Container started
② Cek status	<code>docker-compose ps</code>	Kedua service Up
③ Verifikasi ping+get	<code>docker exec -it risk_api python -c "..."</code>	PING: True GET: w10d02_works
④ Inspect network	<code>docker network inspect <project>_default</code>	Kedua container terdaftar

7. Hal Penting yang Harus Diingat

depends_on BUKAN jaminan readiness. Seperti memastikan pintu ruang Redis sudah dibuka, tapi bukan berarti resepsionis di dalam sudah siap. Ada jeda antara container berjalan dan process Redis siap menerima koneksi. Di development: biarkan restart: unless-stopped handle retry. Di production: gunakan healthcheck (W10D05).

Orphan container. Kalau nama service berubah di Compose file, container lama jadi orphan — masih hidup tapi tidak dikelola. Bersihkan dengan: `docker-compose up -d --remove-orphans`

version: '3.9' obsolete. Docker Compose V2 tidak lagi membutuhkan field version. Hapus baris ini untuk menghindari warning.

8. Knowledge Check — Jawaban

Q1: Kenapa `REDIS_HOST=redis` bukan IP?

Karena 'redis' adalah service name yang di-resolve secara otomatis oleh embedded DNS Docker. IP internal container bisa berubah setiap kali container di-recreate — hardcoded IP akan langsung rusak. Service name selalu konsisten selama nama di Compose file tidak berubah.

Q2: `depends_on` memastikan apa?

Hanya memastikan container Redis sudah started sebelum API container dijalankan. Tidak menjamin Redis process di dalamnya sudah siap menerima koneksi (healthy). Solusi: healthcheck di W10D05.

Q3: `redis:7-alpine` vs `redis:7`?

Alpine menggunakan base image Alpine Linux yang sangat minimal. Hasilnya image Redis hanya ~30MB vs ~110MB untuk non-alpine. Lebih cepat pull, lebih kecil storage, attack surface lebih kecil — semua keuntungan, tidak ada kerugian untuk use case ini.